Lecture 10 — The Team Reference Document and a Full World Finals Simulation

> World Finals Level · Prerequisites: worldfinals-01 (WF format and strategy), worldfinals-02 to 09 (the algorithms), nationals-01 (training system, notebook basics) · Estimated time: 90 min lecture + one 5-hour simulation + 1 h post-mortem

1. Overview

This final lecture turns the whole track into contest performance. It has three parts:

1. **The team reference document (TRD)** — what goes into 25 printed pages, how to lay them out, and how to make sure every line is correct (testing, hashing, a reproducible build).
2. **A full 5-hour World Finals simulation** — a written plan for roles, timeline, keyboard discipline and endgame, plus an **original 12-problem mock set** covering Lectures 2–9, with summarised statements, difficulty ratings and solution sketches.
3. **The post-mortem** — a template that turns the simulation log into concrete action items.

Rules change between years; always check the current ICPC World Finals rules page for the exact TRD limits (page count, paper size, font size) and the announced contest environment.

2. Building the TRD

2.1 Selection criteria

The TRD is not a textbook. During the contest nobody has time to *read* it; it is a source of **pre-verified keystrokes**. A snippet earns its place only if all of the following hold:

1. **Frequency** — the technique has appeared in WF / Div 1 E–F level problems more than once.
2. **Recall cost** — a team member cannot write it bug-free from memory in under ten minutes (Dinic: maybe; blossom, link-cut tree, 3D hull, polynomial exp: no).
3. **Tested and practised** — there is a recorded test, and someone on the team has typed it from paper at least once in practice.

Leave out what every member writes from memory (BFS, Dijkstra, DSU, basic segment tree), long theory, and anything only one person understands.

2.2 Page budget

Section	Pages	Contents (lecture)
Setup	0.5	template, compile flags, `stress.sh`, `hash.sh` (this lecture)
Math and number theory	2	ext. gcd, CRT, Miller–Rabin, Pollard rho, discrete log, Lucy, Du's sieve (L7)
Polynomials	2.5	NTT, double FFT with splitting, inverse/log/exp/sqrt, Berlekamp–Massey, Lagrange interpolation (L2)
Linear algebra and LP	1.5	Gaussian elimination mod p and real, determinant, XOR basis, simplex (L2, L3)
Graphs and matching	4	Dinic, min-cost flow, flow with demands, Hopcroft–Karp, Hungarian, blossom, 2-SAT, bridges/BCC, global min cut (L3)
Trees	2.5	HLD, centroid decomposition, link-cut tree, virtual tree (L4)
Data structures	3	lazy segment tree, Li Chao, implicit treap, persistent segment tree, rollback DSU (for offline connectivity, L5)
Strings	2.5	SA + LCP, suffix automaton, Z, Aho–Corasick, Manacher, eertree, Duval, hashing (L8)
Geometry	4	2D primitives, hull, half-plane intersection, Minkowski sum, circles, polygon cut, 3D primitives, 3D hull, Delaunay (L6)
DP and misc	1.5	CHT, D&C optimisation, Aliens skeleton, slope trick, SOS / subset convolution (L9)
Tables	1	NTT-friendly primes, primes near powers of ten, highly composite numbers, small Catalan/Bell/partition values, identities

The budget is deliberately tight. If a section overflows, cut the snippet that was used least in the last three months of practice.

2.3 Layout

- **Landscape, two or three columns**, monospace at 7–8 pt. A snippet never crosses a column or page break.
- **Fixed section order** with a one-line table of contents on page 1; every member knows "geometry starts on page 17".
- **Snippet header**: name, complexity, one-line usage, preconditions, hash. Preconditions are where contest bugs live: coordinate bounds, 0- vs 1-indexing, whether multi-edges/self-loops are allowed, behaviour on empty input.
- **Code style**: short identifiers, no blank lines, comments only for non-obvious invariants and preconditions; one consistent style across the team so anyone can debug anyone's code.

- **Build it from source**. Keep snippets as compilable `.cpp` files and generate the PDF with a script (KACTL uses LaTeX; `pandoc` or `enscript` also work). Never hand-edit the PDF — the printed code must be byte-identical to the tested code.

2.4 Testing every snippet

Each snippet gets a test file and a verification record (date, method, result). Choose the strongest available method:

Snippet type	Verification
Algorithm with an obvious brute force	randomised stress test with tiny and degenerate inputs
Data structure	random operation sequences against a naive array implementation
Standard library-style problem	accepted submission on an online judge (table below)
Geometry	structural checkers (e.g. 3D hull: every directed edge has its reverse, all points inside, $V - E + F = 2$) on coordinates in $[-3, 3]$
Floating-point code	compare against a `long double` or exact rational version

Verification problems we recommend (all on Library Checker): `convolution_mod`, `inv_of_formal_power_series`, `log_of_formal_power_series`, `exp_of_formal_power_series`, `find_linear_recurrence`, `matrix_det`, `assignment` (Hungarian), `bipartitematching`, `general_matching`, `two_sat`, `scc`, `dynamic_tree_vertex_add_path_sum` (link-cut tree), `line_add_get_min` (Li Chao), `suffixarray`, `number_of_substrings`, `zalgorithm`, `enumerate_palindromes`, `lyndon_factorization`, `runenumerate`, `counting_primes`, `sum_of_totient_function`, `subset_convolution`. Lectures 6–9 of this track contain the stress tests we used for the 3D hull, Delaunay, Minkowski sums, Burnside, sublinear sieves, eertree, runs, SOS, broken profile, Aliens and slope trick.

Run all tests in CI (or a pre-print script) with `-Wall -Wextra -fsanitize=address,undefined`. A snippet without a passing test is not printed.

2.5 Hashing retyped code

Typos when retyping from paper are the most common TRD failure. Print a short hash next to each snippet (and next to every ~10-line block of long snippets). After typing, hash the file and compare: a mismatch localises the typo to one block. The hash ignores whitespace and comments, so formatting differences do not matter.

2.6 Maintenance

Keep the TRD in a git repository (`src/<section>/*.cpp`, `tests/`, `build.sh`, `CHANGELOG`). After every simulation, apply the post-mortem actions: add what was missing, fix what was wrong, cut what was never used. **Freeze** the TRD two weeks before the Finals and use only the printed copy in the last simulations, so that the document you bring is the document you practised with.

3. Reference implementations

3.1 Contest template

```
#include <bits/stdc++.h>
using namespace std;
#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);   // malformed input crashes instead of silently reading
zeros
    int n;
    cin >> n;
    vi a(n);
    for (auto& x : a) cin >> x;
    cout << accumulate(all(a), 0LL) << '\n';
}
```

Debug compile line for the first submission of a hard problem: `g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=address,undefined -D_GLIBCXX_DEBUG sol.cpp`.

3.2 Hash script

```
#!/usr/bin/env bash
# Hash a snippet ignoring whitespace and comments. Usage: ./hash.sh < snippet.cpp
cpp -dD -P -fpreprocessed | tr -d '[:space:]' | md5sum | cut -c-6
```

3.3 Stress-test script and generator

```
#!/usr/bin/env bash
# Usage: ./stress.sh sol brute gen [iterations]
```

```
set -e
for f in "$1" "$2" "$3"; do g++ -std=c++17 -O2 -o "$f" "$f.cpp"; done
for ((i = 1; i <= ${4:-1000}; i++)); do
    ./"$3" "$i" > in.txt
    ./"$1" < in.txt > out1.txt
    ./"$2" < in.txt > out2.txt
    if ! cmp -s out1.txt out2.txt; then
        echo "Mismatch on test $i"; cat in.txt; echo "--- sol:"; cat out1.txt; echo "---
brute:"; cat out2.txt
        exit 1
    fi
done
echo "All tests passed"
```

```
#include <bits/stdc++.h>
using namespace std;
int main(int argc, char** argv) {
    mt19937 rng(atoi(argv[1]));          // seed = test number: every failure is
reproducible
    int n = rng() % 8 + 1;               // small n: brute force is fast and bugs are
readable
    printf("%d\n", n);
    for (int i = 0; i < n; i++) printf("%d ", (int)(rng() % 21) - 10);
    puts("");
}
```

All three were run as shown (template compiled with `-Wall`, a Kadane solution stress-tested against an O(n²) brute force for 300 seeds, and the hash confirmed to be unchanged by adding a comment).

4. The 5-hour simulation

4.1 Setup

- Three contestants, **one computer**, the printed TRD, the judge, nothing else. An **observer** logs: who is at the keyboard (with timestamps), every submission and verdict, and every problem switch.
- Replicate the announced contest environment (OS image, editors, compiler and flags).
- Problem set: the mock set below, or an unseen past World Finals set from the ICPC archive. Never use a set any member has seen.
- Scoreboard frozen for the last hour, as at the Finals. For the mock set, the observer releases "first solve" information to imitate a live scoreboard (e.g. A at 0:09, C at 0:21, H at 0:34).

4.2 Written plan (agreed the day before)

Time	Plan
0:00	Member 1 types the template and scripts, checks hashes; members 2 and 3 start reading
0:00–0:20	Split by letters (A–D, E–H, I–L); for each problem write: topic, difficulty 1–5, estimated lines, key constraint
0:20	Two-minute stand-up: each names their easiest problem; the easiest overall goes to the keyboard
0:20–1:30	Easy problems in increasing order of estimated time; others read the remaining problems in depth
1:30–3:00	Mediums; at most one person at the keyboard, the other two think in a pair or debug from printouts
3:00	Checkpoint: list open problems with estimated remaining time and probability; drop the bottom half
4:00	Freeze: commit to at most two problems, no new reading
4:45	Submit anything plausible; final checks of output format

4.3 Keyboard discipline

- Nobody sits down without a complete plan on paper: algorithm, complexity, data layout, edge cases.
- After a rejected submission: print the code and leave the keyboard. Debug on paper while someone else types.
- Ten-minute rule: if you are stuck at the keyboard for ten minutes, swap.
- Pre-submission checklist: samples; a maximum-size random test for time; n = 1, empty, all equal; 64-bit overflow; output format.
- Long TRD snippets are typed with the hash check (§2.5) before any debugging of the surrounding code.

5. Original mock set (12 problems)

Difficulty is rated for a WF-level team: ★ (everyone solves) to ★★★★★ (few teams). Statements are summaries; write full statements, samples and tests before running the simulation (the sketches below define the intended solutions).

A. Lighthouse Keepers ★. $n \le 2 \cdot 10^5$ lighthouses at positions x_i with ranges r_i illuminate $[x_i - r_i, x_i + r_i]$. Find the minimum number to switch on so that the coast $[0, L]$ is fully lit, or report impossible. *Sketch:* classic interval cover. Sort intervals by left end; repeatedly, among intervals starting at or before the current covered point, take the one reaching farthest. O(n log n). *Traps:* touching intervals count as covering; 64-bit endpoints.

B. Proper Necklaces ★★. Count necklaces of $n \le 10^9$ beads in $k \le 10^9$ colours, up to rotation, with adjacent beads of different colours, modulo $10^9 + 7$. *Sketch:* Burnside (Lecture 7 §2.4): (1/n) Σ_{d | n} φ(n/d) [(k−1)^d + (−1)^d (k−1)]. Factor n by trial division, generate divisors with φ; $n < 10^9 + 7$, so n is invertible. *Traps:* n = 1 gives 0 (a single bead is adjacent to itself); the sign term for odd d; k = 1.

C. Palindrome Diary ★★. A string of length $n \le 5 \cdot 10^5$ arrives character by character; after each character print the number of distinct palindromic substrings so far. *Sketch:* eertree (Lecture 8); the answer is the node count minus 2. O(n·σ) memory with arrays ($26 \cdot 5 \cdot 10^5$ ints ≈ 52 MB) or linked edge lists. *Trap:* computing the new node's link after inserting the edge.

D. Smooth Skyline ★★★. Given heights $a_1..a_n$ ($n \le 2 \cdot 10^5$, $|a_i| \le 10^9$) and $D \ge 0$, choose integers b with $|b_i - b_{i-1}| \le D$ minimising $\Sigma|a_i - b_i|$. *Sketch:* dp_i(x) = |x − a_i| + min_{|y − x| ≤ D} dp_{i−1}(y). Slope trick (Lecture 9): for each i > 1 call `slideMin(-D, D)`, then `addAbs(a_i)`; answer `minf`. O(n log n). *Trap:* applying the window before the first element (harmless) or using the window with swapped signs (wrong).

E. Crystal Facets ★★★. n ≤ 3000 points with integer coordinates $|c| \le 10^6$, not all coplanar. Output the number of (polygonal) faces of their convex hull. *Sketch:* $O(n^2)$ incremental 3D hull with `__int128` predicates (Lecture 6), then merge adjacent triangles whose normals are parallel and equally oriented (exact cross-product test) with a DSU over triangles via a map from directed edges to faces. Answer = number of DSU components. *Traps:* coplanar input points inside a face; duplicates; floating-point normals.

F. Festival Budget ★★★★. Split a sequence of $n \le 2 \cdot 10^5$ positive costs (each $\le 10^4$) into exactly k contiguous groups minimising Σ (group sum)². *Sketch:* the cost w(i, j) = (S_j − S_i)² satisfies the quadrangle inequality (w(a,c) + w(b,d) − w(a,d) − w(b,c) = 2(S_d − S_c)(S_a − S_b) ≤ 0), so the optimum is convex in k. Aliens trick (Lecture 9) with penalty λ per group; the inner DP dp(i) = min_j dp(j) + (S_i − S_j)² + λ is a monotone convex hull trick (slopes −2S_j decreasing, queries S_i increasing) in O(n). Total O(n log C). *Traps:* tie-breaking on the group count must be consistent *inside the hull* (compare (value, count) pairs); λ ranges over [0, S_n^2]; with $S_n \le 2 \cdot 10^9$ the DP values fit in 64 bits, but the hull's intersection comparisons multiply two such quantities and need `__int128`.

G. Visible Orchard ★★★★. Trees stand at all lattice points (x, y), 1 ≤ x ≤ N, 1 ≤ y ≤ M, N, M ≤ 10^{10}. Count those visible from the origin (gcd(x, y) = 1) modulo 998244353. *Sketch:* Σ_d μ(d) ⌊N/d⌋ ⌊M/d⌋ over O(√N + √M) floor blocks (Lecture 7 §3); each block needs a Mertens value M(r), computed by Du's sieve with a linear sieve up to ~$5 \cdot 10^6$ and memoisation — all arguments are of the form ⌊N/i⌋ or ⌊M/i⌋. *Traps:* the two-variable block boundary; overflow of ⌊N/d⌋·⌊M/d⌋ (reduce modulo first).

H. Skill Teams ★★. m ≤ 10^6 workers, each with a skill mask over n ≤ 20 skills. Count non-empty sets of workers whose combined skills (OR) are all n skills, modulo $10^9 + 7$. *Sketch:* c(S) = number of workers with mask ⊆ S (subset zeta, Lecture 9). Sets with OR ⊆ S: 2^{c(S)} − 1. Inclusion–exclusion: Σ_S (−1)^{n−|S|} (2^{c(S)} − 1). $O(n\, 2^n + m)$.

I. Shifting Alliances ★★★★. $n \le 10^5$ cities, $q \le 3 \cdot 10^5$ online operations (decoded with the previous answer): add a road with weight w between two cities in different components; remove an existing road; ask for the maximum road weight on the path between u and v, or -1 if disconnected. *Sketch:* link-cut tree (Lecture 4) with one LCT node per road carrying its weight and path-maximum aggregates; `link`/`cut` through the edge node; `connected` via `findroot`. $O((n + q) \log n)$ amortised. *Traps:* forgetting to push reversal flags before rotating; mapping road ids to edge nodes for deletion.

J. Echoes in the Log ★★★★. A string s ($|s| \le 2 \cdot 10^5$) and $q \le 2 \cdot 10^5$ queries $[l, r]$: does $s[l..r]$ contain a square (a non-empty substring of the form xx)? *Sketch:* a square of half-length t lies in a run of smallest period $p \le t$, and every length-$2p$ window of that run is a square; so the answer is yes iff some run (a, b, p) (half-open) satisfies $\min(b, r + 1) - \max(a, l) \ge 2p$. Compute all runs ($< n$, Lecture 8 §5.3). For each start position x let $\text{best}(x) = \min p$ over runs with $a \le x \le b - 2p$: process runs in increasing p and "paint" unpainted positions of $[a, b - 2p]$ with a DSU skip-pointer, $O(n\,\alpha(n))$. Let $e(x) = x + 2 \cdot \text{best}(x)$ and take suffix minima $E(l) = \min_{x \ge l} e(x)$. Answer yes iff $E(l) \le r + 1$. $O(n \log n + q)$. *Traps:* inclusive/exclusive indices; runs found only in one of the two Lyndon orders.

K. Depot Assignment ★★★. $n \le 500$ trucks and n depots on a grid; cost of assigning truck i to depot j is the Manhattan distance plus a given fee f_{ij}. Find the minimum-cost perfect assignment. *Sketch:* Hungarian algorithm (Lecture 3), $O(n^3) \approx 1.25 \cdot 10^8$. *Trap:* using min-cost flow with an $O(n^2 \log n)$ Dijkstra per augmentation is also fine, but SPFA-based MCMF may time out.

L. Coin Collectors ★★★★★. For each value $v \le n$ ($n \le 2 \cdot 10^5$) there are c_v distinct coin types of value v, with unlimited coins of each type. For every $s = 1..n$, count the multisets of coins of total value s, modulo 998244353. *Sketch:* the answer is $\prod_v (1 - x^v)^{-c_v} = \exp(\sum_v c_v \sum_{k \ge 1} x^{vk}/k)$ (Euler transform, Lecture 7 §4). Build the inner series in $O(n \log n)$ by iterating multiples (harmonic sum), then one power-series exp (Lecture 2), $O(n \log n)$. *Traps:* modular inverses of k up to n; c_v up to 10^9 must be reduced modulo p.

5.1 Expected distribution

A gold-level team should solve 8–10 of these in five hours. A, C, H and K are the "fast four" (all under 40 minutes each for a prepared team) and B follows once the Burnside formula is recalled. D, E, F and G separate medals: each is a known technique with a demanding implementation. I, J and L are the tail; solving one of them is a strong result, and choosing *which* (the one whose snippet is in your TRD) is part of the test.

6. Post-mortem template

Write it within one hour of the end, with the observer's log on the table.

Per problem

Field	
Read at / by	
Difficulty estimate at reading vs actual	
First correct idea at / by	
Coding start / end, coder	
Submissions (time, verdict)	
Root cause of each rejection	
Minutes lost and where (thinking, coding, debugging, idle keyboard)	
What would have caught it earlier	

Team level

1. Timeline chart: keyboard occupancy by person; idle stretches over 5 minutes.
2. Which problem did we misjudge most, and why?
3. Which TRD snippet was missing, wrong, hard to find or slow to type?
4. Which bug classes appeared (overflow, indexing, precision, I/O, misread statement)?
5. One thing to start, one to stop, one to keep.
6. At most five action items, each with an owner and a deadline.

Discuss processes, not people: "the WA on D was found by the observer's max test after 30 minutes" leads to "always run a max test before the first submission", which is actionable.

7. Common mistakes

1. **TRD as a textbook** — pages of theory nobody reads during the contest, and too little code.
2. **Untested snippets** copied the night before; the first time they run is at the Finals.
3. **Hand-edited PDF** drifting from the tested source.
4. **No hashes**: a single typo in a 120-line LCT costs 40 minutes.
5. **Mixed coding styles** — nobody can debug a teammate's printed code.
6. **Simulations without a written plan or an observer**, so there is nothing to compare against.
7. **Endgame drift**: starting a fresh problem at 4:30 instead of finishing one that is 80% debugged.
8. **Changing the TRD in the final week.**

8. Practice problems

- Run the mock set of §5 as a full simulation; then one unseen past World Finals set per month from the ICPC World Finals problem archive.
- Verify TRD snippets on Library Checker (problem ids in §2.4).
- WF problems from this track to re-solve under time pressure with the printed TRD: Panda Preserve (WF 2018), Airport Construction (WF 2017), Asteroids (WF 2015).
- Technique checks for the mock set: CSES 2209 — Counting Necklaces (B), APIO 2014 Palindromes (C), CF 713C (D), CF 449D (H), IOI 2016 Aliens (F), Library Checker — Run Enumerate (J), Library Checker — Exp of Formal Power Series (L).

9. Further reading

- KACTL — KTH's team notebook, an excellent model for layout, hashing and testing: https://github.com/kth-competitive-programming/kactl
- The ICPC World Finals rules and environment pages (current year): https://icpc.global/worldfinals/rules
- Library Checker (verification problems): https://judge.yosupo.jp/
- Codeforces blog posts by World Finals medalists on team strategy and notebooks (search "ICPC team notebook" and "team strategy" on codeforces.com/blog).